

COMPUTATIONAL CONSIDERATIONS IN LARGE SCALE DIGITAL SCENE SIMULATION

Ian J. Curington
Floating Point Systems, UK Ltd,
Apex House, London Road
Bracknell, Berkshire, England.

ABSTRACT

Issues are raised behind why rendering images with a high degree of realism or scene complexity require high computational accuracy and large directly-addressable memory. To efficiently structure scene-simulation algorithms, computer resources must match these requirements. The Floating Point System FPS-264 scientific computer provides both large real memory and full 64-bit precision for efficient simulation of complex scenes.

INTRODUCTION

Rendering images from computer models of three dimensional objects poses special problems when either a high degree of realism or scene complexity is required. Currently there is a great need for techniques that produce high-resolution images of scenes comprised of up to one million polygons, with elaborate lighting and atmospheric models. These algorithms are nearly as complex as the scenes they render, and techniques for improving computational efficiency are still an active research topic. Although some of the techniques are similar to those used to render scenes of lower complexity (such as for CAD and Solid Modeling), higher computational accuracy and larger directly addressable memory is required [1]. These requirements are found in several specific areas, discussed below.

MEMORY REQUIREMENTS

In areas of scene simulation such as flight simulation, a pipeline approach has been used successfully, where polygons are passed in a stream from processor to processor [2]. Successive operations transform, clip, and shade each polygon, with visibility determined at scan line or pixel resolution. This type of system can be made real-time, and can render any number of polygons since they are processed independently from one another, typically spooled from a disk. However, this method makes anti-alias techniques difficult.

The traditional methods of rendering solids in programs such as MOVIE-BYU, MAGI-Synthevision, EUCLID and PATRAN require all objects to be 'in-core' at once. Although these methods can handle large numbers of polygons, they access them in somewhat random order. Thus the amount of real memory required is a function of the size of the model. Some systems produce polygon descriptions 'on-the-fly' within procedural models, which again creates polygons in an arbitrary order. Most software systems of

this type use a rendering algorithm that traverses the polygon list in arbitrary order with many passes. The extension to larger models is often accomplished by using a virtual memory system. Virtual memory provides an easy implementation, but is not the most optimal in performance.

A more fundamental issue concerning random access to polygons concerns anti-aliasing techniques. The problem of avoiding artifacts such as 'jaggies' or interference patterns can only be solved by extending the rendering process beyond the simple pixel point sampling used in most CAD systems. This problem is briefly described in the following quote:

"In any anti-aliasing scheme, the contribution of all the visible objects (even small ones) within a certain distance from the center of the pixel being evaluated need to be taken into account. The final pixel color determined is then a combination of the visible colors or intensities." [3]

In most systems using anti-aliasing techniques, the polygons are traversed in either screen coordinate order or pixel order, but not in polygon generation order. Because the anti-aliasing process includes a larger-than-pixel-size filter shape, one small polygon edge has effects on several adjacent pixels. It is not sufficient to simply over sample and take an average. In polygon pipeline systems, polygons are processed one at a time, so the required information is lost in the average. It cannot generally be known ahead of time in what order the polygons are traversed, hence a large directly addressable storage area for the entire polygon list is needed. Some anti-aliasing systems can use object space partitions to reduce this need, but not in general.

PRECISION REQUIREMENTS

Simulation of scenes with complexity on the order of one million polygons also requires extended arithmetic precision over techniques used for simpler scenes. The reason extended arithmetic precision is necessary is similar to why extended precision is required in large finite difference algorithms. Perspective projections are used for a more realistic look over orthographic projections common in engineering applications. The perspective projection coupled with a large number of polygons introduces the requirement for extended precision [4,5].

The polygon depth or distance from the view point is transformed by a function that behaves similarly to the expression

$$Z_{\text{perspective}} = Z_{\text{max}} - 1/Z$$

Typically, polygons are distributed throughout the model space, so they are usually farther from the view point than near by. After

the perspective transformation, the depth values are compressed, asymptotically approaching Z_{max} as the depth Z increases. Visibility is determined in part by relative depth, or depth differences. The relationship of object depth to perspective depth is shown in figure 1.

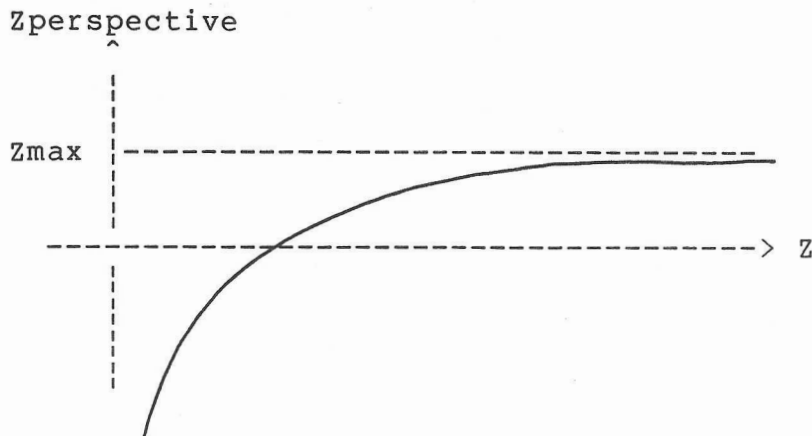


Figure 1. Perspective Transformation effects Depth values.

As more values are compared at great distances, smaller differences between large numbers must be made. This generates the requirement for extended precision. The general relationship between transformations, scene complexity, and precision requirements is shown in figure 2.

	Number of Polygons in Scene	
	10K	1 Million
Orthographic Projection	Single Precision Fixed Point	Single Precision Floating Point
Perspective Projection	Single Precision Floating Point	Extended Precision Floating Point

Figure 2. Depth Precision Requirements.

Additional problems requiring extended precision are the calculation of surface normals for specular reflections, and polynomial root finding in object intersections. Highlights or bright spot reflections from object surfaces are computed by a separate term in most lighting models. The highlight values are a function of polygon surface normal angle, light source direction normal, and viewer direction angle. These three direction vectors are compared, and the cosine of the difference angle raised to a

high power. The typical formulation is similar to

intensity = (COS (alpha))**25,
where alpha = angular difference of surface reflection
direction, and viewer direction.

If there is a numerical error in the surface normal calculations, the highlights tend to jitter or jump around when animated.

In other types of scene rendering, complex lighting effects such as the reflections of one object inside another, refraction through semi-transparent objects, and shadows from multiple light sources are modelled. The most common algorithm to account for these effects is called Ray Tracing. The computationally intensive part of Ray Tracing is not the lighting model, but the intersection calculation of a parametric line with the objects in the scene. This calculation is straight forward algebra for polygons, but is more difficult for other objects.

Calculating the point of intersection between a parametric line and objects that have an algebraic basis can be reduced to solving for the real roots of a polynomial [6,7]. The order of this polynomial depends strongly on the type of object being rendered. Some examples are shown below:

Object type	Degree of Polynomial
Polygon	1
Sphere	2
Toroid	4
Bi-Cubic patch	6
Swept Sphere on	
b-spline path	10
Swept Cubic patch	18

To solve anything higher than third order polynomials over a finite interval requires an indirect, iterative method. The stability of root finding methods at degree 10 or higher is extremely difficult to achieve, even with very high numerical precision. The stability of the root finding method is essential since any error results in an incorrectly shaded pixel that is visible. Usually, 64-bit floating point precision is required in this type of algorithm.

TOWARD AN IDEAL ARCHITECTURE

The FPS-264 family of scientific computers are designed specifically for solving problems where large computational loads, large amounts of memory, and high precision are needed. The directly-addressable physical memory of the FPS-264 can be

expanded in 256k-word increments to 4.5 million words of 64 bits. The pipelined arithmetic section of the FPS-264 supports 15 decimal digits of accuracy. Full software support in FORTRAN easily solves large ill-posed numerical problems.

SUMMARY

Large amounts of memory and extended arithmetic precision are required to produce images where both realism and high scene complexity converge. The areas of perspective depth comparisons, anti-aliasing, and polynomial root finding are shown to have specific computational requirements. The FPS-264 Scientific Computer is well balanced to solve these problems and represents a powerful tool for digital scene simulation.

REFERENCES

- [1] Demos, Brown, and Weinberg, "Digital Scene Simulation: The Synergy of Computer Technology and Human Creativity", Proceedings of the IEEE, Vol 72, No. 1; January 1984.
- [2] Foley and Van Dam, "Fundamentals of Interactive Computer Graphics" Addison Wesley, 1982.
- [3] Weinberg, R. "Parallel Processing Image Synthesis and Anti-Aliasing" Computer Graphics Vol. 15, No. 3, August 1981.
- [4] Whitted, Turner "Image Synthesis Tutorial Notes" Eurographics 1984, Copenhagen.
- [5] Whitted, Turner "Processing Requirements for Hidden Surface Elimination and Realistic Shading" Digest of Papers, COMPCON Spring '82, IEEE 1982.
- [6] Van Wijk, J. J., "Ray Tracing Objects Defined by Sweeping a Sphere", Eurographics '84 Proceedings, Copenhagen, North-Holland publisher.
- [7] Hanrahan, P. "Ray Tracing Algebraic Surfaces", Computer Graphics Vol 17, No. 3, 1982, pp. 245-254.